# Exercícios de SQL Server: criar/alterar tabelas e consultas

Material de estudo com duas listas de exercícios progressivos, baseado nos comandos já vistos: `SELECT FROM`, `DISTINCT`, `WHERE`, `COUNT`, `ORDER BY`, `BETWEEN`, `LIKE`, `SUM`, `MIN`, `MAX`, `AVG`, `GROUP BY`, `HAVING`, `INNER JOIN`, `CREATE TABLE`, `INSERT INTO`, `ALTER TABLE`, `UPDATE` e `DELETE`.

---

## O raciocínio por trás das listas

1. **Um conceito novo por vez.** Cada exercício acrescenta no máximo uma ideia nova às que você já domina. Se travar, o problema é sempre só aquela peça nova.
2. **Os comandos antigos voltam sempre.** O `WHERE` aparece de novo nos exercícios de `UPDATE`, `GROUP BY` e `JOIN`. Repetir em contextos diferentes é o que grava o comando na memória.
3. **Ordem de dependência.** Primeiro você lê os dados (`SELECT`), depois filtra, depois resume (agregações), depois agrupa, depois junta tabelas. Cada etapa precisa da anterior.
4. **Erros de propósito.** Em vários exercícios você deve provocar um erro (inserir duplicado, apagar um registro referenciado). Ler a mensagem de erro ensina como o banco protege os dados.
5. **Dois cenários separados.** A Lista 1 usa uma *Biblioteca*, que você constrói do zero. A Lista 2 usa uma *Loja de informática*, que já vem pronta num script. Assim você pratica consultas com dados reais sem depender de ter feito a Lista 1 perfeita.
6. **Fechamento com desafios.** O último nível de cada lista mistura tudo, sem dica de qual comando usar. É o treino de decidir sozinho.

Novidades pontuais que aparecem e que não estavam na lista de comandos vistos: `PRIMARY KEY`, `IDENTITY`, `NOT NULL`, `DEFAULT`, `UNIQUE`, `CHECK`, `FOREIGN KEY`, `DROP TABLE` e `AS` (apelido). Cada uma entra de forma isolada e ajuda a reforçar `CREATE TABLE`.

---

# Lista 1: Criar e alterar tabelas

**Preparação** (rode uma vez):

```
CREATE DATABASE Biblioteca;
GO
USE Biblioteca;
GO
```

Para recomeçar uma tabela: `DROP TABLE NomeDaTabela;`

### Nível 1: Primeira tabela e primeiros dados

1. Crie a tabela `Livros` com `LivroId` (INT), `Titulo` (VARCHAR(100)), `Autor` (VARCHAR(80)) e `AnoPublicacao` (INT).
2. Insira um livro informando todas as colunas.
3. Insira mais 3 livros, um `INSERT` para cada.
4. Use `SELECT * FROM Livros` para conferir.
5. Crie a tabela `Membros` com `MembroId` (INT), `Nome` (VARCHAR(100)), `Email` (VARCHAR(100)) e `DataCadastro` (DATE).
6. Insira 3 membros (data no formato `'2026-10-08'`).

## Nível 2: Variações do INSERT

7. Insira um livro informando só `Titulo` e `Autor` (liste as colunas). Faça um `SELECT` e veja o que aparece nas outras colunas.
8. Insira 3 livros de uma vez, com um único `INSERT` e vários grupos de valores separados por vírgula.
9. Insira um membro listando as colunas em ordem diferente da tabela (`Email`, `Nome`, ...).

## Nível 3: UPDATE e DELETE

10. Corrija o `AnoPublicacao` de um livro específico.
11. Troque o e-mail de um membro específico.
12. Altere o `Autor` de todos os livros de um mesmo autor (várias linhas de uma vez).
13. Apague um livro pelo `LivroId`.
14. Apague todos os livros com `AnoPublicacao` nulo (`IS NULL`).
15. Crie uma tabela de teste `Lixo` com 3 linhas. Antes de rodar, escreva o que você acha que acontece com `UPDATE` e depois com `DELETE` **sem** `WHERE`. Rode e confira.

## Nível 4: Restrições (regras da tabela)

16. Apague `Livros` com `DROP TABLE` e recrie com `LivroId INT PRIMARY KEY`. Tente inserir dois livros com o mesmo id e leia o erro.
17. Recrie com `LivroId INT IDENTITY(1,1) PRIMARY KEY`. Insira livros **sem** informar o id e veja o número se formando sozinho.
18. Adicione `NOT NULL` em `Titulo` (recriando a tabela). Tente inserir sem título.
19. Acrescente a coluna `Disponivel BIT NOT NULL DEFAULT 1`. Insira um livro sem informar essa coluna.
20. Recrie `Membros` com `MembroId` como `IDENTITY` + `PRIMARY KEY` e `Email` com `UNIQUE`. Tente cadastrar o mesmo e-mail duas vezes.
21. Adicione `CHECK (AnoPublicacao >= 1000)` em `Livros`. Tente inserir o ano 50.

## Nível 5: ALTER TABLE

22. Adicione a coluna `Genero` (VARCHAR(50)) em `Livros`.

23. Adicione a coluna `Telefone` (VARCHAR(20)) em `Membros`.
24. Use `UPDATE` para preencher o `Genero` de todos os livros.
25. Aumente `Titulo` para VARCHAR(200) com `ALTER TABLE ... ALTER COLUMN`.
26. Remova a coluna `Telefone` com `DROP COLUMN`.
27. Adicione `NumeroPaginas INT NOT NULL DEFAULT 0` em `Livros`.
28. Adicione depois uma regra com `ADD CONSTRAINT` para `NumeroPaginas >= 0`. Teste inserindo um valor negativo.

### Nível 6: Relacionamentos (chave estrangeira)

29. Crie `Emprestimos` com `EmprestimoId` (IDENTITY, PK), `LivroId` (NOT NULL), `MembroId` (NOT NULL), `DataEmprestimo` (DATE) e `DataDevolucao` (DATE, aceita nulo). Declare as duas `FOREIGN KEY ... REFERENCES`.
30. Insira 3 empréstimos válidos.
31. Tente inserir um empréstimo com `LivroId` que não existe e leia o erro.
32. Tente apagar um livro que tem empréstimo. Leia o erro e resolva apagando primeiro o empréstimo.
33. Registre uma devolução com `UPDATE` na `DataDevolucao` de um empréstimo (dica: `GETDATE()`).

### Nível 7: Desafios (sem dicas)

34. Crie `Autores` (`AutorId` IDENTITY PK, `Nome`, `Nacionalidade`) e insira 4 autores.
35. **Refatoração:** em `Livros`, adicione `AutorId`, crie a `FOREIGN KEY` para `Autores`, preencha com `UPDATE` e remova a coluna antiga `Autor`.
36. Crie `Editoras` do zero e ligue com `Livros`, seguindo o mesmo padrão, sem olhar os exercícios anteriores.
37. **Mini-projeto:** modele um tema seu (ex.: um sistema de estoque) com pelo menos 3 tabelas relacionadas, restrições e dados de exemplo.

---

# Lista 2: Consultas

**Script de preparação** (rode inteiro uma vez):

```
CREATE DATABASE Loja;
GO
USE Loja;
GO

CREATE TABLE Categorias (
    CategoriaId INT PRIMARY KEY,
    Nome VARCHAR(50) NOT NULL
);
CREATE TABLE Produtos (
    ProdutoId INT PRIMARY KEY,
```

```
    Nome VARCHAR(80) NOT NULL,
    Preco DECIMAL(10,2) NOT NULL,
    Estoque INT NOT NULL,
    CategoriaId INT NOT NULL REFERENCES Categorias(CategoriaId)
);
CREATE TABLE Clientes (
    ClienteId INT PRIMARY KEY,
    Nome VARCHAR(80) NOT NULL,
    Cidade VARCHAR(60),
    Estado CHAR(2),
    DataCadastro DATE
);
CREATE TABLE Pedidos (
    PedidoId INT PRIMARY KEY,
    ClienteId INT NOT NULL REFERENCES Clientes(ClienteId),
    DataPedido DATE NOT NULL,
    Status VARCHAR(20) NOT NULL
);
CREATE TABLE ItensPedido (
    ItemId INT IDENTITY(1,1) PRIMARY KEY,
    PedidoId INT NOT NULL REFERENCES Pedidos(PedidoId),
    ProdutoId INT NOT NULL REFERENCES Produtos(ProdutoId),
    Quantidade INT NOT NULL
);

INSERT INTO Categorias VALUES (1,'Periféricos'),(2,'Hardware'),(3,'Redes'),(4,'Armazenamento'

INSERT INTO Produtos VALUES
(1,'Mouse Gamer',89.90,50,1),
(2,'Teclado Mecânico',249.90,30,1),
(3,'Monitor 24 pol',799.00,15,1),
(4,'Webcam HD',159.90,0,1),
(5,'Placa de Vídeo',2399.00,5,2),
(6,'Memória RAM 8GB',189.90,40,2),
(7,'Processador i5',1199.00,12,2),
(8,'Fonte 500W',279.90,20,2),
(9,'Roteador Wi-Fi',199.90,25,3),
(10,'Cabo de Rede 5m',19.90,100,3),
(11,'SSD 480GB',229.90,35,4),
(12,'HD Externo 1TB',349.90,18,4);

INSERT INTO Clientes VALUES
(1,'Ana Souza','São José dos Campos','SP','2025-01-15'),
(2,'Bruno Lima','Campinas','SP','2025-03-02'),
(3,'Carla Mendes','Rio de Janeiro','RJ','2025-03-20'),
(4,'Diego Alves','São José dos Campos','SP','2025-06-10'),
(5,'Eduarda Rocha','Belo Horizonte','MG','2025-08-05'),
(6,'Felipe Santos','Campinas','SP','2026-01-12'),
(7,'Gabriela Nunes','Rio de Janeiro','RJ','2026-02-28'),
(8,'Henrique Costa','Curitiba','PR','2026-04-17');

INSERT INTO Pedidos VALUES
(1,1,'2026-01-20','Entregue'),
```

```
(2,2,'2026-02-05','Entregue'),
(3,1,'2026-03-11','Entregue'),
(4,3,'2026-04-02','Cancelado'),
(5,4,'2026-05-18','Entregue'),
(6,5,'2026-06-09','Enviado'),
(7,2,'2026-07-21','Entregue'),
(8,6,'2026-08-14','Pendente'),
(9,1,'2026-09-03','Enviado'),
(10,7,'2026-09-25','Pendente');

INSERT INTO ItensPedido (PedidoId, ProdutoId, Quantidade) VALUES
(1,1,1),(1,2,1),(2,5,1),(3,6,2),(3,11,1),(4,3,1),(5,9,1),(5,10,5),
(6,7,1),(6,8,1),(7,1,2),(7,12,1),(8,4,1),(9,11,2),(9,6,2),(10,2,1);
```

*Se os acentos aparecerem estranhos, troque `'texto'` por `N'texto'` e use `NVARCHAR` no lugar de `VARCHAR`.*

## Nível 1: SELECT e FROM

1. Mostre todos os dados de `Clientes`.
2. Mostre só `Nome` e `Cidade` dos clientes.
3. Mostre `Nome` e `Preco` dos produtos.
4. Repita o 3 com apelidos: `Nome AS Produto`, `Preco AS Valor`.

## Nível 2: DISTINCT e ORDER BY

5. Liste as cidades distintas dos clientes.
6. Liste os estados distintos.
7. Liste os produtos do mais barato ao mais caro.
8. Liste os produtos do mais caro ao mais barato ( `DESC` ).
9. Liste os clientes ordenados por `Estado` e, dentro de cada estado, por `Nome`.

## Nível 3: WHERE

10. Clientes do estado SP.
11. Produtos com preço maior que 200.
12. Produtos sem estoque.
13. Pedidos com status diferente de 'Entregue' ( `<>` ).
14. Clientes de SP **ou** RJ.
15. Produtos da categoria 1 **e** com preço menor que 200.
16. Pedidos 'Pendente' ou 'Enviado', ordenados pela data.

## Nível 4: BETWEEN e LIKE

17. Produtos com preço entre 100 e 300.
18. Pedidos feitos entre `'2026-03-01'` e `'2026-06-30'`.

19. Clientes cadastrados em 2025 (usando `BETWEEN` com datas).
20. Clientes cujo nome começa com 'A'.
21. Produtos cujo nome termina com 'GB'.
22. Clientes cujo nome tem 'a' como segunda letra ( `'_a%'` ).
23. Produtos com preço entre 50 e 300 **e** nome começando com 'M'.

## Nível 5: Funções de agregação

24. Quantos clientes existem ( `COUNT` )?
25. Quantos produtos existem?
26. Quantos clientes são de SP?
27. Qual o estoque total ( `SUM` )?
28. Qual o menor e o maior preço ( `MIN` , `MAX` )?
29. Qual o preço médio ( `AVG` )? Dê um apelido à coluna.
30. Em **um único** `SELECT` : quantidade, menor, maior e média de preço dos produtos da categoria 2.
31. Quantas cidades diferentes há entre os clientes ( `COUNT(DISTINCT ...)` )?
32. Qual o valor total do estoque ( `SUM(Preco * Estoque)` )?

## Nível 6: GROUP BY

33. Quantidade de clientes por estado.
34. Quantidade de pedidos por status.
35. Quantidade de produtos e preço médio por `CategoriaId` .
36. Estoque total por `CategoriaId` , do maior para o menor.
37. Quantidade de pedidos 'Entregue' por `ClienteId` (o `WHERE` vem antes do `GROUP BY` ).

## Nível 7: HAVING

38. Estados com mais de 1 cliente.
39. Categorias ( `CategoriaId` ) com preço médio acima de 200.
40. Clientes com 2 ou mais pedidos.
41. Pedidos não cancelados por cliente, só de quem tem 2 ou mais, ordenados do maior para o menor. ( `WHERE` + `GROUP BY` + `HAVING` + `ORDER BY` juntos.)

## Nível 8: INNER JOIN

42. Produtos com o **nome** da categoria ( `Produtos` + `Categorias` ).
43. Pedidos com o nome do cliente.
44. Produtos com a categoria, ordenados por categoria e depois por preço.
45. Pedidos de clientes de SP, mostrando nome do cliente, data e status.
46. Produtos da categoria 'Redes' (filtre pelo **nome** da categoria).
47. Cada item de pedido com o nome do produto e a quantidade.

## Nível 9: JOIN + agregação

48. Quantidade de produtos por **nome** de categoria.
49. Quantidade de pedidos por **nome** de cliente.
50. Valor total de cada pedido ( `SUM(Quantidade * Preco)` agrupado por `PedidoId` ).
51. Total gasto por cliente, ignorando pedidos cancelados (4 tabelas juntas).
52. Categorias cujo total vendido passa de 500.

### Nível 10: Desafios (sem dicas)

53. Qual cliente mais gastou?
54. Qual produto vendeu mais unidades?
55. Qual o ticket médio (valor médio por pedido)?
56. **Extra:** quais produtos nunca foram vendidos? (Vai exigir `LEFT JOIN` , que ainda não foi visto. Pesquise e tente.)

---

## Como usar para gravar de verdade

- **Escreva cada comando à mão**, sem copiar e colar.
- **Preveja o resultado antes de rodar** (quantas linhas, quais valores). O erro de previsão é onde você aprende.
- **Faça por níveis**: um nível por dia e, no dia seguinte, refaça 3 ou 4 exercícios do nível anterior de memória.
- **Se travar mais de 10 minutos**, volte ao exercício anterior em vez de procurar a resposta.

---

# Anexo: isso tudo é SQL Server da Microsoft?

Sim. Tudo foi escrito para o **SQL Server da Microsoft**, que usa a variante de SQL chamada **T-SQL** (Transact-SQL). Alguns pontos são específicos dele:

- `IDENTITY(1,1)` para numeração automática. No MySQL seria `AUTO_INCREMENT` , e no PostgreSQL, `SERIAL` .
- `GO` separando blocos de comandos. Ele não é SQL de verdade, é um separador que o SSMS (SQL Server Management Studio) entende.
- `USE NomeDoBanco` para escolher o banco ativo.
- `BIT` para valores verdadeiro/falso (0 ou 1).
- `GETDATE()` para a data e hora atuais.
- `ALTER TABLE ... ALTER COLUMN` para mudar o tipo de uma coluna. No MySQL seria `MODIFY COLUMN` .
- `DATE` , `DECIMAL(10,2)` , `VARCHAR` e `CHAR` existem em quase todos os bancos, mas os detalhes variam um pouco.
- `TOP` , típico do SQL Server para pegar só as primeiras linhas. No MySQL e no PostgreSQL seria `LIMIT` .

O que vale em qualquer banco, porque faz parte do SQL padrão:

- `SELECT`, `FROM`, `WHERE`, `DISTINCT`, `ORDER BY`
- `COUNT`, `SUM`, `MIN`, `MAX`, `AVG`
- `GROUP BY`, `HAVING`, `INNER JOIN`
- `INSERT`, `UPDATE`, `DELETE`
- `CREATE TABLE`, `PRIMARY KEY`, `FOREIGN KEY`, `UNIQUE`, `CHECK`, `DEFAULT`, `NOT NULL`

Então a lógica de consulta (Lista 2) vale para qualquer banco com pouquíssimas mudanças. Na Lista 1 (criar e alterar tabelas) é onde aparecem mais diferenças de sintaxe entre os bancos.